Micronsoft Solutions (Pvt) Ltd

Software Engineering Internship | Recruitment Division

Subject: Practical Technical Assessment - Software Engineering Internship

---

Dear Candidate,

Thank you for your interest in joining the engineering team at **Micronsoft Solutions (Pvt) Ltd**. We have reviewed your application and would like to invite you to complete this technical assessment.

At Micronsoft Solutions, we value logic, scalability, and optimized performance. This assessment is designed to test your ability to handle real-world challenges including concurrency, high-volume data, and transactional integrity.

Timeframe: Please submit your solution within **48 hours** of receiving this document.

Preferred Tech Stack: Python (Django) + React.js (Database: MySQL).

Challenge 01: High-Concurrency Management

Scenario:

You are managing a high-demand flash sale for a product with exactly **50 units** in stock. Within seconds, 500+ concurrent requests hit the server to purchase this item.

- **Task:** Build a Django API endpoint POST /api/purchase/.
- **Requirement:** Ensure stock never drops below zero, even with identical millisecond requests.
- **Proof:** Provide a Python test script (using threading or asyncio) that fires 100 concurrent requests to verify your logic.

Challenge 02: Big Data Aggregation & Query Optimization

Scenario:

You need to generate an analytics dashboard for a system handling massive transaction volume.

- **Task:** Write a script to populate the database with **100,000 dummy transaction records** from the last 6 months.
- **Requirement:** Create an API GET /api/analytics/ that returns daily revenue for 30 days and the Top 5 products.
- **Performance:** The API must respond in **under 500ms** despite the 100k records.

Challenge 03: Mini POS System (State & Transactional Integrity)

Scenario:

We need a simple, fast Point of Sale interface for cashiers to process daily sales.

- **Frontend:** Create a single-page POS UI. It should have two main sections: a product selection area and a shopping cart. The cashier should be able to click on products to add them to the cart, adjust quantities, and see the real-time Grand Total.
- **Backend:** Create a checkout API endpoint (POST /api/checkout/) that accepts the entire cart payload (e.g., list of items, quantities, and total amount).
- **Integrity:** You MUST use transaction.atomic. If any single item fails to save, the entire order must rollback.
- **Bonus:** Style a "Receipt Component" to fit a standard **80mm Thermal Printer** width.

****

Submission Guidelines:

1. **Repository**: Push your complete code (Frontend, Backend, and the Load Test Script) to a public GitHub repository.
2. **Documentation:** Include a detailed README.md file explaining:
    i. How to set up and run the project locally.
    ii. How to run the data population script.
    iii. A brief explanation of the approach you took to solve the concurrency issue, optimize the database queries, and handle the POS transactions.
3. **Reply:** Reply directly to this email with the link to your GitHub repository.

If you have any technical difficulties setting up the environment, please feel free to reach out.

Good luck!